BizConnect — Complete Codebase Reference

Firestore Collections · Firebase Config · Auth · Coins · Components · Cloud Functions · Security Rules

```
// ========== FIRESTORE COLLECTIONS MASTER ==========

// 1. Auth & Identity
users: {
  id: string, // auth.uid
  email: string,
  phone: string,
  passwordHash: string, // if using email/pass
  twoFactorEnabled: boolean,
  vipAdminCode: string,
  createdAt: timestamp
}

profiles: {
  userId: reference(users.id),
  username: string,
  name: string,
  bio: string,
  profilePictureUrl: string,
  followersCount: number,
  followingCount: number,
  likesCount: number,
  referralCode: string, // e.g. "OBI002"
  referralLink: string, // https://bizconnect.app/signup?ref=OBI002
  createdAt: timestamp,
  updatedAt: timestamp
}

// 2. Content
posts: {
  id: string,
  userId: reference(users.id),
  contentUrl: string, // video/image
  caption: string,
  giftShopEnabled: boolean,
  giftCardEnabled: boolean,
  createdAt: timestamp,
  updatedAt: timestamp
}

recordings: {
  id: string,
  userId: reference(users.id),
  mode: "Live" | "Gallery" | "Livestream",
  duration: number,
  musicId: string,
  createdAt: timestamp
}

stories: {
  id: string,
  userId: reference(users.id),
  postId: reference(posts.id) | null,
  streamId: reference(channelStreams.id) | null,
  createdAt: timestamp
```

```
}

// 3. Social / Messaging
messages: {
  id: string,
  senderId: reference(users.id),
  receiverId: reference(users.id),
  content: string,
  translatedContent: string,
  createdAt: timestamp,
  deletedForSender: boolean,
  deletedForReceiver: boolean,
  options: ["Edit","Delete","Translate","Forward","Reply","VoiceNote","AudioCall","VideoCall"]
}

voiceNotes: {
  id: string,
  senderId: reference(users.id),
  receiverId: reference(users.id),
  audioUrl: string, // Firebase Storage
  duration: number,
  language: string,
  voiceStyle: string,
  createdAt: timestamp
}

calls: {
  id: string,
  callerId: reference(users.id),
  receiverId: reference(users.id),
  type: "audio" | "video",
  status: "ringing" | "accepted" | "ended",
  startedAt: timestamp,
  endedAt: timestamp
}

// 4. Monetization
gifts: {
  id: string,
  name: string,
  type: "GiftShop" | "GiftCard",
  coinCost: number
}

giftTransactions: {
  id: string,
  senderId: reference(users.id),
  receiverId: reference(users.id),
  giftId: reference(gifts.id),
  createdAt: timestamp
}

coinWallets: {
  userId: reference(users.id),
  balance: number,
  updatedAt: timestamp
}

coinTransactions: {
  id: string,
  userId: reference(users.id),
```

```
  type: "Gift" | "Download" | "MusicCommission" | "Stream" | "Referral",
  amount: number,
  createdAt: timestamp
}

// 5. Channels / Live
channels: {
  id: string,
  ownerId: reference(users.id),
  name: string,
  type: "SportsAnalysis" | "General",
  createdAt: timestamp
}

channelStreams: {
  id: string,
  channelId: reference(channels.id),
  videoUrl: string,
  duration: number,
  coinCostPerMinute: number,
  createdAt: timestamp
}

// 6. Growth
referrals: {
  id: string,
  userId: reference(users.id),
  code: string,
  referralLink: string,
  createdAt: timestamp
}

referralTransactions: {
  id: string,
  referrerId: reference(users.id),
  newUserId: reference(users.id),
  code: string,
  referrerReward: number, // 2000
  newUserReward: number, // 1000
  createdAt: timestamp
}

// 7. Admin
adminActions: {
  id: string,
  adminId: reference(users.id),
  action: string,
  targetUser: reference(users.id),
  details: object,
  createdAt: timestamp
}

// ========== firebase.js ==========
import { initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";
import { getFirestore, serverTimestamp, increment, arrayUnion } from "firebase/firestore";
import { getStorage } from "firebase/storage";

const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_PROJECT.firebaseapp.com",
```

```
  projectId: "YOUR_PROJECT_ID",
  storageBucket: "YOUR_PROJECT.appspot.com",
  messagingSenderId: "YOUR_SENDER_ID",
  appId: "YOUR_APP_ID"
};

const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const db = getFirestore(app);
export const storage = getStorage(app);
export { serverTimestamp, increment, arrayUnion };

// ========== auth.service.js ==========
import { auth, db, serverTimestamp, increment } from "./firebase";
import { createUserWithEmailAndPassword, signInWithEmailAndPassword, signOut } from "firebase/auth";
import { doc, setDoc, getDocs, collection, query, where, addDoc, updateDoc } from "firebase/firestore";

export const signUp = async (email, password, username, referralCode = null) => {
  const { user } = await createUserWithEmailAndPassword(auth, email, password);

  const code = username.toUpperCase().slice(0,3) + Math.floor(Math.random()*1000);
  const link = `https://bizconnect.app/signup?ref=${code}`;

  await setDoc(doc(db, "users", user.uid), { email, createdAt: serverTimestamp() });
  await setDoc(doc(db, "profiles", user.uid), {
    userId: user.uid, username, referralCode: code, referralLink: link,
    followersCount: 0, followingCount: 0, likesCount: 0, createdAt: serverTimestamp()
  });
  await setDoc(doc(db, "coinWallets", user.uid), {
    userId: user.uid, balance: 0, updatedAt: serverTimestamp()
  });

  if (referralCode) {
    const q = query(collection(db, "profiles"), where("referralCode", "==", referralCode));
    const snap = await getDocs(q);
    if (!snap.empty) {
      const referrerId = snap.docs[0].id;
      await addDoc(collection(db, "referralTransactions"), {
        referrerId, newUserId: user.uid, code: referralCode,
        referrerReward: 2000, newUserReward: 1000, createdAt: serverTimestamp()
      });
      await updateDoc(doc(db, "coinWallets", referrerId), { balance: increment(2000), updatedAt: serverTimestamp() });
      await updateDoc(doc(db, "coinWallets", user.uid), { balance: increment(1000), updatedAt: serverTimestamp() });
    }
  }
  return user;
};

export const login = (email, password) => signInWithEmailAndPassword(auth, email, password);
export const logout = () => signOut(auth);

// ========== coins.service.js ==========
import { db, increment, serverTimestamp } from "./firebase";
import { doc, updateDoc, addDoc, collection } from "firebase/firestore";

export const spendCoins = async (userId, amount, type) => {
  await updateDoc(doc(db, "coinWallets", userId), {
    balance: increment(-amount), updatedAt: serverTimestamp()
  });
  await addDoc(collection(db, "coinTransactions"), {
    userId, amount: -amount, type, createdAt: serverTimestamp()
```

```
  });
};

// ========== Cloud Functions (index.js) ==========
const functions = require("firebase-functions/v2");
const admin = require("firebase-admin");
const { Translate } = require('@google-cloud/translate').v2;
const OpenAI = require("openai");
const { AccessToken } = require('livekit-server-sdk');

admin.initializeApp();
const db = admin.firestore();
const translate = new Translate();
const openai = new OpenAI({ apiKey: process.env.OPENAI_KEY });

// A. SECURE COIN SPEND
exports.spendCoins = functions.https.onCall(async (request) => {
  const { userId, amount, type } = request.data;
  if (request.auth.uid !== userId) throw new functions.https.HttpsError('permission-denied');

  const walletRef = db.doc(`coinWallets/${userId}`);
  return db.runTransaction(async (t) => {
    const doc = await t.get(walletRef);
    const balance = doc.data().balance;
    if (balance < amount) throw new functions.https.HttpsError('failed-precondition', 'Not enough coins');

    t.update(walletRef, { balance: admin.firestore.FieldValue.increment(-amount), updatedAt: admin.firestore.FieldValue.ser
    t.create(db.collection('coinTransactions').doc(), { userId, amount: -amount, type, createdAt: admin.firestore.FieldValu
    return { success: true, newBalance: balance - amount };
  });
});

// B. TRANSLATE DM TEXT
exports.translateMessage = functions.https.onCall(async (request) => {
  const { messageId, targetLang } = request.data;
  const msgRef = db.doc(`messages/${messageId}`);
  const msg = (await msgRef.get()).data();
  if (msg.senderId !== request.auth.uid && msg.receiverId !== request.auth.uid)
    throw new functions.https.HttpsError('permission-denied');

  const [translation] = await translate.translate(msg.content, targetLang);
  await msgRef.update({ translatedContent: translation });
  return { translatedContent: translation };
});

// C. LIVEKIT TOKEN
exports.getLiveKitToken = functions.https.onCall((request) => {
  const { room, identity, name } = request.data;
  const at = new AccessToken(process.env.LIVEKIT_API_KEY, process.env.LIVEKIT_API_SECRET, { identity, name });
  at.addGrant({ roomJoin: true, room, canPublish: true, canSubscribe: true });
  return { token: at.toJwt() };
});

// ========== HomeFeed.jsx ==========
import { useState, useEffect } from "react";
import { db } from "../services/firebase";
import { collection, getDocs } from "firebase/firestore";
import { spendCoins } from "../services/coins.service";

export default function HomeFeed({ userId }) {
  const [posts, setPosts] = useState([]);
```

```
  useEffect(() => {
    const fetchPosts = async () => {
      const snap = await getDocs(collection(db, "posts"));
      setPosts(snap.docs.map(d => ({ id: d.id, ...d.data() })));
    };
    fetchPosts();
  }, []);

  const handleDownload = async (post) => {
    await spendCoins(userId, 50, "Download");
    window.open(post.contentUrl, "_blank");
  };

  return (
    <div className="home-feed">
      {posts.map(post => (
        <div className="content-card" key={post.id}>
          <video src={post.contentUrl} controls width="100%" />
          <p>{post.caption}</p>
          <div className="options">
            {post.giftShopEnabled && <button>Gift Shop</button>}
            {post.giftCardEnabled && <button>Gift Card</button>}
            <button>Comment</button>
            <button>Share</button>
            <button>Save</button>
            <button onClick={() => handleDownload(post)}>Download (50c)</button>
          </div>
        </div>
      ))}
    </div>
  );
}

// ========== VoiceNote.jsx ==========
import { useState, useRef } from "react";
import { db, storage, serverTimestamp } from "../services/firebase";
import { ref, uploadBytes, getDownloadURL } from "firebase/storage";
import { collection, addDoc } from "firebase/firestore";

export default function VoiceNote({ senderId, receiverId }) {
  const [recording, setRecording] = useState(false);
  const mediaRecorderRef = useRef(null);
  const chunks = useRef([]);

  const startRecording = async () => {
    const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
    mediaRecorderRef.current = new MediaRecorder(stream);
    mediaRecorderRef.current.ondataavailable = e => chunks.current.push(e.data);
    mediaRecorderRef.current.onstop = async () => {
      const blob = new Blob(chunks.current, { type: "audio/webm" });
      const fileRef = ref(storage, `voiceNotes/${Date.now()}.webm`);
      await uploadBytes(fileRef, blob);
      const url = await getDownloadURL(fileRef);
      await addDoc(collection(db, "voiceNotes"), {
        senderId, receiverId, audioUrl: url,
        duration: Math.round(blob.size/1000), language: "en",
        voiceStyle: "male", createdAt: serverTimestamp()
      });
      chunks.current = [];
    };
    mediaRecorderRef.current.start();
```

```
    setRecording(true);
  };

  const stopRecording = () => {
    mediaRecorderRef.current.stop();
    setRecording(false);
  };

  return (
    <div className="voice-note">
      <button onClick={recording ? stopRecording : startRecording}>
        {recording ? "Stop" : "Record"}
      </button>
    </div>
  );
}

// ========== CallOptions.jsx (with LiveKit) ==========
import { Room, RoomEvent, VideoTrack, AudioTrack } from 'livekit-client';
import { getFunctions, httpsCallable } from "firebase/functions";
import { useEffect, useRef, useState } from "react";

export default function LiveKitRoom({ roomName, userId, userName, onLeave }) {
  const [tracks, setTracks] = useState([]);
  const roomRef = useRef(null);

  useEffect(() => {
    const join = async () => {
      const fn = httpsCallable(getFunctions(), 'getLiveKitToken');
      const { data } = await fn({ room: roomName, identity: userId, name: userName });
      const room = new Room();
      roomRef.current = room;
      room.on(RoomEvent.TrackSubscribed, (_track, publication, participant) => {
        setTracks(prev => [...prev, { track: publication.track, participant }]);
      });
      room.on(RoomEvent.TrackUnsubscribed, (_track, publication) => {
        setTracks(prev => prev.filter(t => t.track !== publication.track));
      });
      await room.connect('wss://YOUR_LIVEKIT_URL', data.token);
      await room.localParticipant.enableCameraAndMicrophone();
    };
    join();
    return () => roomRef.current?.disconnect();
  }, []);

  return (
    <div className="livekit-room">
      {tracks.map(({ track, participant }) => (
        track.kind === 'video'
          ? <VideoTrack key={track.sid} track={track} participant={participant} />
          : <AudioTrack key={track.sid} track={track} />
      ))}
      <button onClick={onLeave}>Leave</button>
    </div>
  );
}

// ========== MessageCard.jsx ==========
import { useState } from "react";
import { db } from "../services/firebase";
import { doc, updateDoc, deleteDoc } from "firebase/firestore";
```

```
import VoiceNote from "./VoiceNote";
import CallOptions from "./CallOptions";

export default function MessageCard({ msg, userId }) {
  const [showMenu, setShowMenu] = useState(false);

  const editMessage = async () => {
    const newContent = prompt("Edit:", msg.content);
    if (newContent) await updateDoc(doc(db, "messages", msg.id), { content: newContent });
  };

  const deleteMessage = async (mode) => {
    if (mode === "both") await deleteDoc(doc(db, "messages", msg.id));
    else await updateDoc(doc(db, "messages", msg.id), { deletedForReceiver: true });
  };

  const handleTranslate = async () => {
    const fn = httpsCallable(getFunctions(), 'translateMessage');
    const { data } = await fn({ messageId: msg.id, targetLang: 'fr' });
    alert(`Translated: ${data.translatedContent}`);
  };

  return (
    <div className="message-card">
      <p>{msg.content}</p>
      <button onClick={() => setShowMenu(!showMenu)}>Options</button>
      {showMenu && (
        <div className="menu">
          <button onClick={editMessage}>Edit</button>
          <button onClick={() => deleteMessage("me")}>Delete for Me</button>
          <button onClick={() => deleteMessage("both")}>Delete for Both</button>
          <button onClick={handleTranslate}>Translate</button>
          <button>Reply</button>
          <button>Forward</button>
          <VoiceNote senderId={userId} receiverId={msg.receiverId} />
          <CallOptions callerId={userId} receiverId={msg.receiverId} />
        </div>
      )}
    </div>
  );
}

// ========== ReferralSection.jsx ==========
export default function ReferralSection({ profile }) {
  const copyLink = () => {
    navigator.clipboard.writeText(profile.referralLink);
    alert("Referral link copied! +1000c for new user, +2000c for you");
  };

  return (
    <div className="referral">
      <p>Your Code: {profile.referralCode}</p>
      <button onClick={copyLink}>Share Referral Link</button>
    </div>
  );
}

// ========== Firestore Security Rules ==========
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
```

```
    function signedIn() { return request.auth != null; }
    function isOwner(id) { return request.auth.uid == id; }
    function isParticipant(s, r) {
      return signedIn() && (request.auth.uid == s || request.auth.uid == r);
    }

    match /users/{userId} { allow read, update: if isOwner(userId); allow create: if signedIn(); }
    match /profiles/{userId} { allow read: if signedIn(); allow write: if isOwner(userId); }
    match /coinWallets/{id} { allow read: if isOwner(id); allow write: if false; }
    match /coinTransactions/{id} {
      allow read: if isOwner(resource.data.userId);
      allow create: if isOwner(request.resource.data.userId);
    }
    match /messages/{id} {
      allow read, update, delete: if isParticipant(resource.data.senderId, resource.data.receiverId);
      allow create: if isParticipant(request.resource.data.senderId, request.resource.data.receiverId);
    }
    match /voiceNotes/{id} {
      allow read, create: if isParticipant(request.resource.data.senderId, request.resource.data.receiverId);
    }
    match /calls/{id} {
      allow read, create: if isParticipant(request.resource.data.callerId, request.resource.data.receiverId);
      allow update: if isParticipant(resource.data.callerId, resource.data.receiverId);
    }
    match /posts/{id} { allow read: if signedIn(); allow write: if isOwner(resource.data.userId); }
    match /referrals/{id} { allow read: if signedIn(); allow write: if false; }
    match /{document=**} { allow read, write: if false; }
  }
}

// ========== CSS Styles ==========
.home-feed {
  background: linear-gradient(135deg, #6a11cb, #2575fc);
  min-height: 100vh;
  padding: 20px;
}
.content-card {
  background: #fff;
  border-radius: 20px;
  box-shadow: 0 10px 25px rgba(0,0,0,0.2);
  margin-bottom: 20px;
  padding: 15px;
  transition: transform 0.3s;
}
.content-card:hover { transform: perspective(1000px) rotateY(5deg); }
.options button, .menu button {
  margin: 5px;
  padding: 10px 15px;
  border-radius: 10px;
  border: none;
  background: linear-gradient(90deg, #ff6a00, #ee0979);
  color: white;
  font-weight: bold;
  cursor: pointer;
}
.message-card { background: #f1f1f1; padding: 10px; border-radius: 12px; margin: 8px 0; }

// ========== translateVoiceNote service ==========
export const translateVoiceNote = async (audioUrl, targetLang, voiceStyle) => {
  // 1. STT - get text
  const text = await callWhisperAPI(audioUrl);
```

```
  // 2. Translate text
  const translatedText = await callGPTAPI(text, targetLang);
  // 3. TTS with voice style
  const newAudioUrl = await callElevenLabsAPI(translatedText, voiceStyle);
  // 4. Save to Firestore
  await addDoc(collection(db, "voiceNotes"), {
    ...original, audioUrl: newAudioUrl, language: targetLang,
    translatedFrom: original.audioUrl
  });
  return newAudioUrl;
};
```